

Теория параллелизма: Задание 2

«Параллельная реализация решения системы линейных алгебраических уравнений с помощью OpenMP»

В данном задании требуется реализовать две версии многопоточных программ реализация решения СЛАУ с использованием OpenMP:

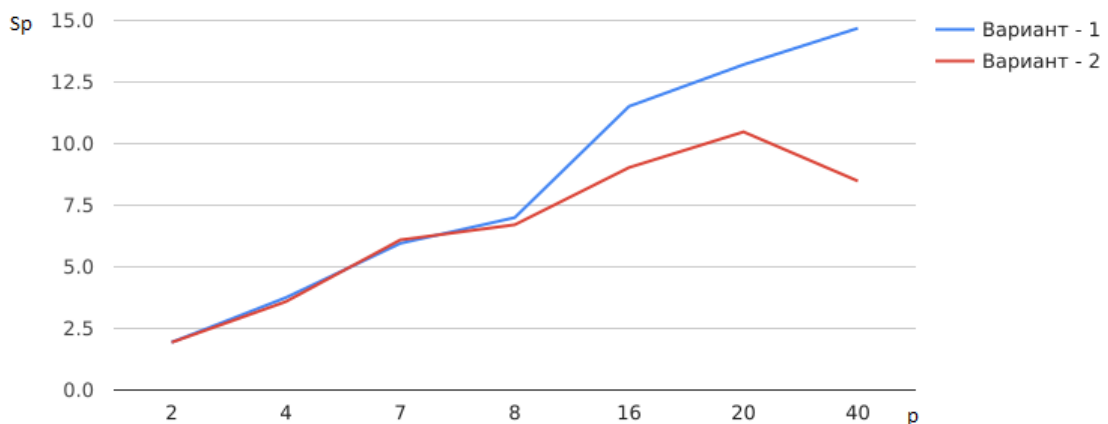
- 1) для каждого распараллеливаемого цикла создается отдельная параллельная секция `#pragma omp parallel for`,
- 2) : создается одна параллельная секция `#pragma omp parallel`, охватывающая весь итерационный алгоритм.

Также необходимо сравнить работу программ при разной политике распределения потоков `schedule` (см. Репозиторий README.md). Анализ масштабируемости кода в разных вариантах программ приведен ниже в виде таблицы, где была использована сетка $256 * 256$:

Вариант	Количество потоков							
	2			4		7		8
	T1	T2	S2	T4	S4	T7	S7	T8
1	30,8823	15,9235	1,939416586	8,22487	3,754746276	5,18653	5,954327845	4,41186
2	37,428	19,3492	1,934	10,4243	3,590456913	6,14087	6,094901862	5,57901

Вариант	Количество потоков					
	16		20		40	
	T16	S16	T20	S20	T40	S40
1	2,68171	11,51589844	2,33836	13,20682016	2,10301	14,6848089
2	4,14382	9,032245609	3,5723	10,47728354	4,41284	8,48161274

Также диаграмма:



Теперь о политике распределения потоков schedule. В данном примере мы использовали политику динамического распределения (Dynamic) и статическое распределение (Static), в зависимости от того, как мы распределяли итерации цикла на каждый поток, время выполнения программы было различным.

Ниже предоставлена краткая сводка на основе запущенной программы второго варианта с сеткой 120*120 и с количеством потоков равным 40:

При N = 120:

- 1) $N / 40 \rightarrow$ static = 1,55; dynamic = 1.40
- 2) $N / 50 \rightarrow$ static = 2,15; dynamic = 1.66

Вывод:

В первом примере мы использовали распределение которое делится без остатка на предоставленные потоки, поэтому статическое распределение было как и динамическое одинаково эффективно, итерации цикла имеют одинаковую или схожую

вычислительную нагрузку и равномерно распределяются между потоками. Также при статическом распределении потоки заранее знают, какие итерации они будут выполнять. Однако динамическое распределение немного опережает статическое, что особенно видно во втором варианте, так как помимо того, что итерации сами по себе имеют сложный характер (из-за вложенности цикла), так и еще неравномерное распределение потоков из-за дополнительных остаточных итераций: $120 / 50 = 2.4$.

Различие в двух вариантах исполнения программ:

- **Параллелизация внешнего уровня против параллелизации внутреннего цикла:** во втором варианте вся итерация `while` параллелизована, но это неэффективно, поскольку внутренняя часть цикла (а не часть вычислений в циклах `for`) выполняется всеми потоками, что приводит к избыточной работе, а директива `#pragma omp parallel for` в первом варианте эффективно параллелизует внешний цикл `for (int i = 1; i < N - 1; i++)`, распределяя итерации по потокам, что значительно ускоряет вычисления.
- **Редукция ошибки:** во втором варианте ошибка (`error`) не редуцирована - обновление переменной `error` выполняется последовательно всеми потоками, что может приводить к некорректным результатам из-за гонок данных. В первом варианте использование `reduction(max:error)` гарантирует корректное и безопасное обновление переменной `error` в параллельной среде. Редукция позволяет правильно вычислить максимальную ошибку среди всех потоков.
- **Отсутствие параллелизации внутренних циклов:** во втором варианте внутренние циклы не параллелизованы, что означает, что они выполняются последовательно. Это

значительно снижает производительность на больших массивах данных.